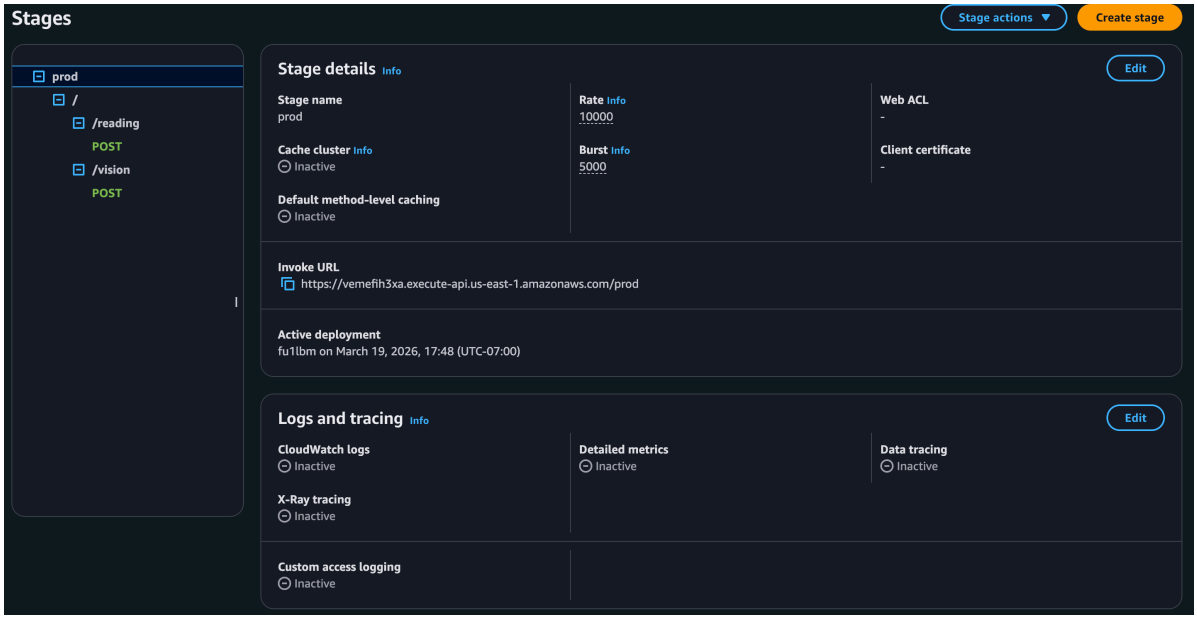
Week 4

Week 4 Status Update — Phase 4

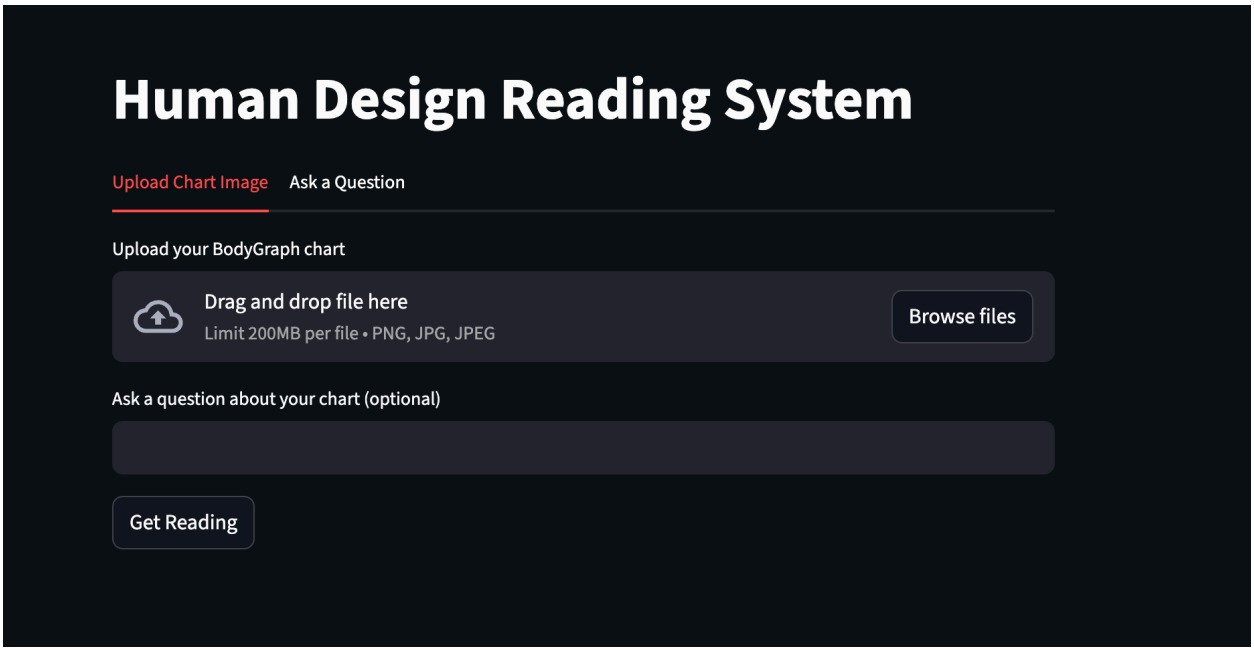
Phase 4: API Gateway + Streamlit Frontend

What was completed

- Created a **REST API** in **API Gateway** (`human-design-api`) with two endpoints:
  - `POST /vision` → triggers `human-design-vision` Lambda (Phase 2)
  - `POST /reading` → triggers `human-design-rag` Lambda (Phase 3)
- Deployed API to `prod` stage
  - Invoke URL: `https://vemefih3xa.execute-api.us-east-1.amazonaws.com/prod`



- Built a **Streamlit** frontend on **EC2** (`t3.micro`) with two tabs:
  - **Upload Chart Image**: upload BodyGraph image → Vision extraction → RAG reading
  - **Ask a Question**: query only → RAG reading
- Streamlit accessible at `http://54.163.49.98:8501`



Architecture decision: API Gateway timeout limitation

API Gateway has a hard **29 sec timeout** limit. Phase 3 RAG Lambda cold start takes ~190 seconds (loading PyTorch + sentence-transformers + S3 download + Anthropic API call), which exceeds this limit.

Solution: Hybrid approach

- `/vision` (Phase 2) → **API Gateway** (fast, completes in ~5 seconds)
- `/reading` (Phase 3) → **boto3** `lambda_client.invoke` directly from Streamlit EC2 (no timeout limit, waits up to Lambda's 5-minute timeout)

In production, this would be solved with **Lambda Provisioned Concurrency** (keeps containers warm) or migrating to **ECS/Fargate** (always-on container, no cold start). Both options were impractical for this student project due to cost constraints.

Services used

- API Gateway (REST API)
- EC2 (`t3.micro`, Streamlit frontend)
- Lambda (invoked via API Gateway and boto3)

Next steps

- Phase 5: CI/CD pipeline with GitHub Actions + Golden Dataset evaluation